

OOP in Python

Object-Oriented Programming

A Complete Lecture Plan with Code Examples

1. Classes & Objects

2. Constructors

3. Attributes & Methods

4. Encapsulation

5. Inheritance

6. Polymorphism

7. Abstraction

8. Mini Project

Introduction to OOP

What is OOP and why does it matter?

What is OOP?

OOP is a programming paradigm that organizes code around objects and classes rather than functions and procedures.

Core 4 Pillars:

Encapsulation • Inheritance
Polymorphism • Abstraction

Why Use OOP?

- Reusability — write once, use many times
- Modularity — break complex problems into objects
- Maintainability — easy to update code
- Scalability — grows with your project
- Real-world modeling — mirrors how we think

Key Concepts

- A class is a blueprint/template
- An object is an instance of a class
- Use class keyword to define
- Objects are created by calling the class like a function
- Each object has its own state
- Multiple objects can share the same class



python

```
class Car:
    brand = "Toyota"
    speed = 0

# Creating objects (instances)
car1 = Car()
car2 = Car()

car1.brand = "BMW"
car2.brand = "Tesla"

print(car1.brand)    # BMW
print(car2.brand)    # Tesla
print(car1.speed)    # 0
```

Constructor `__init__()`

Initializing objects at creation time

Key Concepts

- `__init__` runs automatically when an object is created
- `self` refers to the current object
- Use `self.attr` to set instance attributes
- Constructor can take parameters
- Each object gets its own copy of attributes
- Always define `__init__` for proper initialization



python

```
class Person:
    def __init__(self, name, age):
        self.name = name    # instance attribute
        self.age = age

# Objects initialized at creation
p1 = Person("Alice", 25)
p2 = Person("Bob", 30)

print(p1.name)    # Alice
print(p2.name)    # Bob
print(p1.age)     # 25
```

Attributes and Methods

Data and behavior inside a class

Key Concepts

- Attributes = data stored in an object
- Methods = functions defined inside a class
- self is the first parameter of every method
- Instance attributes are unique per object
- Class attributes are shared across instances
- Call methods using dot notation: obj.method()



python

```
class BankAccount:
    bank = "Python Bank"    # class attribute

    def __init__(self, owner, balance=0):
        self.owner = owner    # instance attr
        self.balance = balance

    def deposit(self, amount):    # method
        self.balance += amount

    def show(self):
        print(f"{self.owner}: $" + "{self.balance}")

acc = BankAccount("Alice", 100)
acc.deposit(50)
acc.show()    # Alice: $150
```

Encapsulation

Hiding internal details from the outside world

Key Concepts

- Controls access to class internals
- Public: accessible from anywhere (name)
- `_protected`: convention, internal use
- `__private`: name-mangled, truly hidden
- Use getters/setters to expose data safely
- Prevents accidental modification of data

```
class Employee:
    def __init__(self, name, salary):
        self.name = name        # public
        self.__salary = salary  # private

    def get_salary(self):        # getter
        return self.__salary

    def set_salary(self, amount): # setter
        if amount > 0:
            self.__salary = amount

emp = Employee("Alice", 5000)
print(emp.get_salary())    # 5000

emp.set_salary(6000)
print(emp.get_salary())    # 6000
# emp.__salary → AttributeError!
```

Child classes inherit from parent classes

Key Concepts

- Child class reuses parent's code
- Syntax: `class Child(Parent)`
- `super()` calls the parent's method
- Child can override parent methods
- Supports multi-level inheritance
- Promotes code reuse — DRY principle

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):      # inherits Animal
    def __init__(self, name, breed):
        super().__init__(name) # call parent
        self.breed = breed

    def speak(self):    # override
        print(f"{self.name} says: Woof!")

d = Dog("Rex", "Labrador")
d.speak()              # Rex says: Woof!
print(isinstance(d, Animal)) # True
```

Polymorphism

Same interface, different implementations

Key Concepts

- poly = many, morphism = forms
- Same method name, different behavior per class
- Method overriding is key to polymorphism
- Works with inheritance hierarchies
- Enables writing generic code
- Python uses duck typing for polymorphism



python

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self):
        return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h): self.w=w; self.h=h
    def area(self):
        return self.w * self.h

shapes = [Circle(5), Rectangle(4, 6)]

for s in shapes:      # same method, diff result
    print(s.area())
# 78.5
# 24
```

Key Concepts

- Hides complexity from the user
- Uses the abc module (Abstract Base Class)
- @abstractmethod marks required methods
- Cannot instantiate abstract classes
- Subclasses must implement all abstract methods
- Enforces a consistent interface / contract



python

```
from abc import ABC, abstractmethod

class Vehicle(ABC):          # abstract class
    @abstractmethod
    def start_engine(self): # must be overridden
        pass

    @abstractmethod
    def stop_engine(self):
        pass

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started")

    def stop_engine(self):
        print("Car engine stopped")

my_car = Car()
my_car.start_engine()    # Car engine started
# Vehicle() → TypeError!
```

Mini Project — Library System

Applying all OOP concepts together

Encapsulation

Inheritance

Polymorphism

Abstraction

__init__

Methods

library_system.py

```
from abc import ABC, abstractmethod

class LibraryItem(ABC):          # Abstraction
    def __init__(self, title, year):
        self.title = title
        self.__year = year      # Encapsulation

    def get_year(self): return self.__year

    @abstractmethod
    def display_info(self): pass  # must be implemented

class Book(LibraryItem):         # Inheritance
    def __init__(self, title, year, author):
        super().__init__(title, year)
        self.author = author

    def display_info(self):       # Polymorphism
        print(f"[Book] {self.title} by {self.author} ({self.get_year()})")

class Magazine(LibraryItem):     # Inheritance
    def __init__(self, title, year, issue):
        super().__init__(title, year)
        self.issue = issue
```

OOP in Python — Summary

You now know the 4 pillars of OOP

Encapsulation

Hide internal data

Inheritance

Reuse parent code

Polymorphism

One interface, many forms

Abstraction

Simplify complexity

Keep practicing with the Mini Project — build more real-world systems!